

토론 에세이 (1)

공과대학 기계공학부 안준영 학번: 2024-11078

들어가며: 실습 기록

이번 과제는 오픈소스 디자인 킷의 탁구 게임기(pong_Uno, 04_Table_Ball_Paddle)를 수업에서 받은 마이크로컨트롤러 키트(ATmega328p + 128×64 OLED)에 직접 빌드·업로드하고, 실행해본 경험을 바탕으로 토론과제를 서술하는 것이다. 실습 과정은 작은 시행착오의 연속이었다. 보드가 컴퓨터에 인식되지 않아 원인을 추적해보니 케이블의 데이터선 문제와 USB-시리얼 변환 칩(CH340) 드라이버 부재가 겹쳐 있었고, 데이터 가능한 케이블 교체와 드라이버 설치, 보드 설정(ATmega328P 5V/16MHz)을 거쳐서야 업로드에 성공했다. 전원이 들어온다고 통신이 되는 것은 아니라는 — 전기 신호의 역할이 나뉜다는 사실을 몸으로 배운 과정이었다.

OLED에 시작 화면이 뜨고 게임이 실행되는 것을 확인했다. 왼쪽에는 벽이 하나 서 있고, 공이 화면 안을 튀어 다니며, 오른쪽 탁구채를 버튼으로 움직여 공을 받아내는 방식이다. 이름은 탁구(pong)이지만 실제 탁구와는 거리가 있었다 — 상대가 있는 경기라기보다 벽에 공을 튕겨 계속 받아내는 1인 게임에 가까웠다.

토론 1. 순차 실행 언어와 병렬 실행 언어

이 게임의 C++ 코드는 위에서 아래로 한 줄씩 실행된다. loop() 안에서는 "패들 지우기 → 버튼 읽기 → 패들 갱신·그리기 → 공 지우기 → 공 갱신·그리기 → 점수 처리 → 화면 전송"이 코드에 적힌 순서 그대로 일어난다. CPU가 한 순간에 하나의 명령만 실행하기 때문이다. 플레이하면서 흥미로웠던 것은, **공은 계속 움직이는데 벽은 가만히 있다**는 점이었다. 코드에서도 벽 그리기는 setup()에서 한번만 호출된다. 순차 실행 언어에서는 "변하지 않는 것은 다시 그릴 이유가 없다"가 자연스럽다 — 한 번 그린 벽은 메모리(캔버스)에 남아 있으니까.

그런데 이 생각을 하드웨어로 옮기면 이야기가 달라진다. 베릴로그로 이 게임을 만든다면 벽·공·패들·충돌 판정·화면 출력은 각각 별도의 물리적 회로로 존재하며 클럭마다 **동시에** 동작한다. "벽을 다시 그리지 않는다"가 아니라 "벽 출력 회로가 매 순간 벽을 출력하고 있다"에 가깝다. 정리하면 소프트웨어는 하나의 실행기가 **시간을 나눠 쓰는** 절차의 언어이고, 하드웨어 언어는 회로를 **공간에 펼쳐 놓는** 병렬의 언어이다. 코드의 위→아래 순서는 후자에서는 실행 순서가 아니라 회로의 배치가 된다. 순차적 사고를 그대로 옮기면 경쟁 상태 같은 함정에 빠진다는 점이 두 세계를 오가며 가장 의식해야 할 차이라고 느꼈다.

토론 2. 래스터 스캔과 VSYNC·HSYNC

래스터 스캔(raster scan)은 화면의 화소를 정해진 순서대로 하나씩 찍어 나가는 방식이다. 왼쪽에서 오른쪽으로 한 줄을 훑고, 다음 줄로 내려가 또 왼쪽에서 오른쪽으로 — 화면 맨 아래까지 반복하면 한 장의 화면이 완성되고, 다시 맨 위로 돌아간다. CRT의 전자빔 주사에서 온 개념이 오늘날 디스플레이 신호에도 이어져 있다.

이 순서를 송신·수신 측이 합의하는 신호가 HSYNC와 VSYNC이다. HSYNC(수평 동기)는 "한 줄이 끝났으니 다음 줄의 시작으로"라는 신호, VSYNC(수직 동기)는 "한 프레임이 끝났으니 맨 위로"라는 신호이다. 화소 데이터에는 좌표가 없기 때문에, 수신 측은 VSYNC 이후 몇 번째 HSYNC인지, HSYNC 이후 몇 번째 화소인지를 세어 위치를 안다 — 타이밍만으로 위치를 합의하는 구조이다. 이번 OLED는 I2C로 프레임버퍼에 통째로 쓰는 방식이라 이 신호들이 쓰이지 않았지만, 학기 후반에 내 칩으로 비디오 신호를 만들 때는 카운터 회로로 주사 위치를 세며 HSYNC/VSYNC를 직접 발생시키게 될 것이다. 소프트웨어의 drawBitmap() 한 줄이 하드웨어에서는 주사 위치 카운터와 동기 신호 발생 회로로 바뀌는 셈이다.

토론 3. 버그 찾기: 탁구채를 통과하는 공

게임을 플레이하며 이상한 점을 발견했다. 공이 탁구채의 모서리에 맞을 때 제대로 반사되지 않고, 패들과 겹쳐 보이는데도 그대로 통과해 버리는 경우가 실제로 있었다. 공이 패들의 윤곽을 따라 튕기는 것이 아니라 뚫고 지나가는 느낌이었다. 원인은 충돌 판정 코드에 있었다:

```
if (xPos == (SCREEN_WIDTH-PADDLE_W-8)
    && yPos < (yPaddle+PADDLE_H)
    && yPos > (yPaddle)) // Hit
    dx = false;
```

공은 8×8 픽셀 비트맵인데, 이 판정은 y 방향으로 공의 꼭짓점(yPos) 하나만 패들 범위와 비교한다. 공의 몸체 8픽셀은 검사에 들어가지 않는다. 예를 들어 yPos = yPaddle - 5이면 공은 yPaddle-5 ~ yPaddle+2에 걸쳐 있어 화면상 분명 패들과 닿아 있지만, 조건 yPos > yPaddle이 거짓이라 충돌로 인식되지 않는다. 플레이에서 목격한 "통과"와 "모서리 반사 불량"이 바로 이 코드의 결과였다. 게다가 부등호가 모두 엄격한 부등호(<, >)라 yPos == yPaddle처럼 경계에 정확히 닿는 경우도 충돌로 치지 않는다.

공의 높이를 포함해 사각형 대 사각형의 겹침으로 고치면 된다:

```
if (xPos == (SCREEN_WIDTH-PADDLE_W-8)
    && (int16_t)yPos + 8 > (int16_t)yPaddle
    && yPos < yPaddle + PADDLE_H)
    dx = false;
```

공의 아랫변($yPos+8$)이 패들 윗변보다 아래에 있고, 공의 윗변($yPos$)이 패들 아랫변보다 위에 있으면 겹친다. 이렇게 고치면 모서리 접촉도 정상적으로 반사된다. 그래픽 충돌 판정은 "점"이 아니라 "면적"으로 생각해야 한다는 것 — 화면에 보이는 물체는 모두 크기를 가지는데 코드가 한 점만 검사하면 눈(직관)과 코드(논리) 사이에 틈이 생기고, 그 틈이 곧 버그가 된다는 것을 배웠다.

토론 4. 소프트웨어로 되는데, 굳이 전용 칩인가

솔직히 처음에는 "소프트웨어로 충분히 되는데 왜 굳이 칩을 만들지?"라는 의문이 있었다 — 이 게임도 상용 마이크로컨트롤러의 소프트웨어로 잘 돌아갔으니까. 그러나 생각보면 이유가 있다. 첫째, **훨씬 적은 컴퓨팅 자원으로 같은 일을 더 효율적으로** 할 수 있다. 범용 CPU는 명령어 인출·해석 같은 부대 회로가 항상 함께 돌지만, 전용 회로는 필요한 게이트만 동작하므로 더 작은 면적과 더 낮은 전력으로 구현된다 — 배터리 기기에서는 곧 사용 시간이다. 둘째, **소프트웨어의 부담을 줄일 수 있다**. 화면 출력·충돌 판정처럼 정해진 일을 반복하는 작업을 전용 하드웨어가 떠맡으면 프로세서는 다른 일을 하거나 아예 필요 없어진다. 하드웨어 가속기가 곁들여진 것이 오늘날 대부분의 칩(그래픽 카드, AI 가속기)의 모습인 이유이다. 셋째, **타이밍의 확실성**이다. 비디오 신호처럼 나노초 단위 타이밍이 필요한 일을 소프트웨어 루프로는 흔들림 없이 만들기 어렵지만, 카운터 회로는 클럭에 정확히 묶인다.

마지막으로 이 수업의 관점에서는, 내가 기술한 알고리즘이 실리콘으로 제작되어 돌아온다는 것 자체가 목적이다. 남의 칩에 코드를 심는 것과, 내 의도를 묘사한 회로가 물리적으로 존재하는 것은 설계자에게 완전히 다른 경험이다.

토론 5. ++yPos와 yPos++

C/C++에서 ++yPos(전위)는 값을 **먼저** 증가시키고 증가된 값을 식의 결과로 내놓고, yPos++(후위)는 현재 값을 **먼저** 내놓은 뒤 증가시킨다. 단독 문장에서는 결과가 같지만 식 안에서는 다르다 ($yPos=3$ 일 때 $a=++yPos$ 는 $a=4$, $b=yPos++$ 는 $b=4$, $yPos=5$). 게임 코드의 $xPos = (dx)? ++xPos : --xPos$;에서는 갱신된 좌표를 즉시 대입해야 하므로 전위형이 의도에 맞는다 — 후위형이었다면 공이 한 박자 늦게 움직였을 것이다. 이런 구분이 성립하는 것은 C++가 순차 실행 언어라서 "증가"와 "사용"의 전후 순서가 언어의 평가 규칙으로 확정되기 때문이다.

하드웨어 언어에서는 이 문제가 다른 차원이 된다. 베릴로그에서는 보통 `always @(posedge clk)` 블록 안에 `yPos <= yPos + 1;`처럼 non-blocking 대입으로 쓰고, "이번 클럭의 옛 값 + 1이 다음 상태로 등록"된다. 모든 문장이 동시에 평가되므로 "증가 전 값을 쓸지 후 값을 쓸지"는 식 안의 순서가 아니라 **할당 방식(blocking = / non-blocking <=)**과 **클럭 에지**, 즉 시간축의 문제가 된다. 베릴로그

계열에 ++, -- 같은 단항 연산을 함부로 두면 위험한 이유다: ++는 blocking 갱신 의미라 시뮬레이션과 합성 결과가 어긋날 수 있고, 병렬 문장들이 같은 레지스터를 갱신하면 순차 언어의 안전장치(한 줄씩 실행)가 없어 경쟁 상태가 생기며, 조합 회로 블록에서는 의도하지 않은 래치가 추론될 수 있다. 순차 언어에서 ++의 전위·후위가 작은 함정이라면, 하드웨어 언어에서 증분 연산의 순서 착각은 **회로가 다르게 만들어지는** 문제로 커진다. 그래서 HDL에서는 다음 상태를 명시적으로 쓰는 코딩 스타일이 권장된다.

맺으며

겉으로는 벽 하나와 튀는 공의 단순한 게임이었지만, 캔버스와 프레임버퍼, 순차와 병렬, 동기 신호, 충돌 판정의 면적까지 하드웨어 설계의 출발점이 되는 개념이 이 작은 코드 안에 모두 들어 있었다. 무엇보다 게임을 하다 느낀 어색함(공의 패들 통과)이 코드 한 줄과 정확히 연결되는 것을 확인했을 때, "설계는 표현이다"라는 말이 조금 실감났다. 다음에는 이 게임이 소프트웨어가 아닌 회로로 표현될 때 무엇이 달라지는지 따라가 보고 싶다.

나아가 이 수업에서 만들어 보고 싶은 것이 있다. 배터리 없이 수확한 극소량의 에너지로 사는 노드들에서조차 범용 프로세서는 명령어 인출·메모리·클럭 트리라는 큰 오버헤드를 짊어진다. 나는 이번 학기의 만 개 트랜지스터 예산 안에서, 그 범용 프로세서를 통째로 빼고 통신 프로토콜 로직을 하드와이어드 회로로만 실행하는 최소 칩을 설계·제작해 보고 싶다. 제약(트랜지스터 만 개)을 받아들이고 그 안에서 창의력으로 함수를 지켜내는 설계 이 게임기에서 시작해 그 칩의 테이프아웃까지 완주하는 것이 이 수업에서의 나의 목표이다.